

The Pocket Guide to Web Development

HTML, CSS, and JavaScript from First Principles

Suraj B.

Table of Contents

HTML: The Structure of the Web	14
Introduction to HTML	15
A first document	15
Tags, elements, and attributes	15
Why HTML matters	15
The HTML Document Structure	16
The <head> element	16
The <body> element	16
The doctype	16
Whitespace and nesting	16
Text Elements: Headings, Paragraphs, and Formatting	17
Headings	17
Paragraphs and line breaks	17
Inline emphasis	17
Inline code and quotations	17
Links and Navigation	18
The href attribute	18
Opening in a new tab	18
Link text matters	18
Navigation	18
Lists: Ordered, Unordered, and Definition	19

Unordered lists	19
Ordered lists	19
Description lists	19
Nesting lists	19
Images and Multimedia	20
The element	20
Responsive images	20
Audio and video	20
The <picture> element	20
Tables	21
Anatomy of a table	21
Captions and scope	21
Spanning cells	21
Forms and Input Types	22
Form structure	22
Input types	22
Other controls	22
Client-side validation	22
Semantic HTML	23
Page landmarks	23
Content sections	23
Why bother	23
A worked example	23
HTML Metadata and the Head	24

Character encoding	24
The viewport meta tag	24
Title and description	24
Open Graph and Twitter Cards	24
Accessibility Basics (a11y)	25
The big wins	25
ARIA	25
Testing	25
It is the law	25
CSS: The Look and Feel	26
Introduction to CSS	27
A rule	27
Where CSS lives	27
The cascade	27
CSS Selectors and Specificity	28
The core selectors	28
Combinators	28
Specificity	28
Practical advice	28
The Box Model	29
The four layers	29
Width and height	29
box-sizing: border-box	29
Margin collapse	29

Display, Position, and Float	30
Display	30
Position	30
Float	30
z-index	30
Flexbox: The Modern Layout Tool	31
The flex container	31
Main and cross axis	31
Growing and shrinking	31
Wrapping	31
CSS Grid: Two-Dimensional Layouts	32
Defining a grid	32
Placing items	32
Auto-placement and minmax	32
Typography in CSS	33
Font family	33
Size, weight, and line height	33
Web fonts	33
Rhythm	33
Colors, Gradients, and Backgrounds	34
Color formats	34
Gradients	34
Background shorthand	34
Multiple backgrounds	34

Transitions and Animations	35
Transitions	35
Keyframe animations	35
Respect user preferences	35
Responsive Design and Media Queries	36
Mobile-first	36
Media query syntax	36
Common breakpoints	36
Fluid units	36
Container queries	36
CSS Variables (Custom Properties)	37
Defining and using	37
Theming	37
Fallbacks	37
JavaScript interop	37
Pseudo-classes and Pseudo-elements	38
State pseudo-classes	38
Structural pseudo-classes	38
Pseudo-elements	38
Why bother	38
JavaScript: The Behavior Layer	39
Introduction to JavaScript	40
Where it runs	40
Adding it to a page	40

Modules	40
Your first script	40
Variables, Data Types, and Operators	41
let, const, and var	41
Primitive types	41
Type coercion	41
Operators worth knowing	41
Control Flow: Conditionals and Loops	42
If/else	42
Ternary	42
Loops	42
break and continue	42
Functions and Scope	43
Declarations vs expressions	43
Arrow functions	43
Default and rest parameters	43
Scope and closures	43
Arrays and Objects	44
Arrays	44
Functional array methods	44
Objects	44
Destructuring and spread	44
The DOM: Selecting and Modifying Elements	45
Selecting elements	45

Reading and writing content	45
Attributes and properties	45
Class and style manipulation	45
Creating and inserting	45
Events and Event Handling	46
addEventListener	46
The event object	46
Bubbling and delegation	46
Removing and aborting	46
Asynchronous JavaScript	47
Callbacks	47
Promises	47
async/await	47
Parallel work	47
ES6+ Features Worth Knowing	48
Template literals	48
Optional chaining and nullish coalescing	48
Object shorthand and computed keys	48
Modules	48
Classes	48
Beyond the Basics	49
Embedded Content: iframe, object, embed	50
The <iframe> element	50
Sandboxing	50

Same-origin and CORS	50
Lazy loading	50
Canvas, SVG, and Drawing	51
When to use SVG	51
When to use Canvas	51
Inline SVG vs external SVG	51
Accessibility	51
Microdata, Schema.org, and SEO	52
JSON-LD	52
Picking the right type	52
Testing	52
Beyond structured data	52
CSS Architecture: BEM, ITCSS, and Utility-First	53
BEM	53
ITCSS	53
Utility-first	53
Picking one	53
CSS Layers, Nesting, and Modern Features	54
@layer	54
Native nesting	54
:has() the parent selector	54
Subgrid	54
Print Stylesheets, Dark Mode, and Media Features	55
Dark mode	55

Print styles	55
Hover, pointer, and motion	55
Color gamut and contrast	55
Strings, Arrays, and Numbers: A Deeper Look	56
String methods worth memorizing	56
Array methods you will use weekly	56
Number quirks	56
Useful number tools	56
Error Handling	57
try/catch/finally	57
Throwing errors	57
Async errors	57
Graceful degradation	57
Working with JSON	58
JSON.parse and JSON.stringify	58
Pretty printing	58
What JSON cannot represent	58
Streaming and large data	58
The Fetch API and HTTP	59
A basic request	59
Why ok matters	59
Sending data	59
Aborting	59
CORS	59

Browser Storage	60
localStorage	60
sessionStorage	60
IndexedDB	60
Cookies	60
Cache API	60
Security Essentials	61
Cross-Site Scripting (XSS)	61
Cross-Site Request Forgery (CSRF)	61
Content Security Policy (CSP)	61
HTTPS everywhere	61
Don't roll your own crypto	61
Performance Basics	62
Measure first	62
Ship less code	62
Images	62
Caching	62
Avoid layout thrashing	62
How the Browser Renders a Page	63
Parsing	63
Style	63
Layout	63
Paint	63
Composite	63

Forms in JavaScript	64
Reading form values	64
preventDefault	64
Validation API	64
Controlled vs uncontrolled	64
Browser DevTools	65
Elements	65
Console	65
Network	65
Performance	65
Application	65
Putting It All Together	66
The HTML shell	66
The CSS foundation	66
The JavaScript entry point	66
Where to go next	66
Closing Notes	67
What to learn next	67
A final word	67
Glossary	68

Preface

This book is a compact field guide to the three core technologies of the modern web: HTML, CSS, and JavaScript. It does not try to be exhaustive. Instead, it explains the small set of ideas you actually use every day, the way a working developer would explain them to a colleague.

Each chapter is short, opinionated, and accompanied by code you can paste into a browser and run. The book assumes you have written a little code before, in any language, but it does not assume any prior web experience.

Read it cover to cover, or skip to the chapter you need. The index at the front lists every topic, and chapters cross-reference each other where relevant. By the end you should be able to read most real-world web code, write your own from scratch, and know what to look up when you get stuck.

Part I

HTML: The Structure of the Web

Introduction to HTML

HTML, short for HyperText Markup Language, is the skeleton of every web page. It does not describe how a page looks, nor what it does. It only describes what the page contains: a heading here, a paragraph there, a list, an image, a link. Visual styling is the job of CSS, and behavior is the job of JavaScript.

A first document

The minimum useful HTML document looks like this. Every browser on earth will understand it, even though there is no styling and no script. Save it as `index.html` and open it in your browser.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Hello</title>
  </head>
  <body>
    <h1>Hello, world</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

Tags, elements, and attributes

A tag is the literal text inside angle brackets, like `<p>`. An element is the entire opening tag, content, and closing tag together. An attribute is extra metadata attached to the opening tag, written as `name="value"`. The `lang` attribute on `<html>`, for example, tells screen readers and search engines what language the page is in.

Why HTML matters

HTML is the contract between your content and every browser, assistive technology, and search engine that will ever read it. Writing it carelessly is the single most common cause of inaccessible, unsearchable, or unmaintainable websites. Treat HTML as documentation, not as decoration.

The HTML Document Structure

Every HTML document is a tree of nested elements with a single root: the `<html>` element. Inside it live two children, `<head>` and `<body>`. Understanding this structure makes everything else easier.

The `<head>` element

The head holds information about the page that is not displayed in the body of the document. This includes the title shown in the browser tab, links to stylesheets, character encoding, viewport settings, and metadata for search engines and social previews.

```
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>My Page</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

The `<body>` element

Everything the user can see lives inside `<body>`: headings, paragraphs, images, links, forms, and so on. The body is rendered top to bottom in normal flow, which means each block element stacks below the previous one.

The doctype

The very first line of every modern HTML document is `<!doctype html>`. This single line tells the browser to use the modern, standards-compliant rendering mode rather than its legacy compatibility mode. Forgetting it is a classic source of subtle layout bugs.

Whitespace and nesting

HTML collapses consecutive whitespace into a single space when rendering. You can indent and break lines however you like in the source without changing the output. Indent consistently, because HTML's only protection against bugs is your eyes.

Text Elements: Headings, Paragraphs, and Formatting

Text is the bulk of most web pages. HTML gives you a small, well-defined vocabulary for marking it up. Use the right tag for the right meaning and the rest of the platform, from accessibility to search, rewards you for free.

Headings

There are six levels of headings, from `<h1>` to `<h6>`. They are not about font size; they are about document outline. Use exactly one `<h1>` per page for its main subject, then `<h2>` for major sections, `<h3>` for subsections, and so on. Skipping levels confuses screen readers.

```
<h1>The Pocket Guide to Web Development</h1>
<h2>Part I: HTML</h2>
<h3>Chapter 1: Introduction</h3>
```

Paragraphs and line breaks

Wrap each paragraph of prose in `<p>`. To force a single line break without starting a new paragraph, use `<br>`. Avoid `<br>` for visual spacing; that is what CSS margins are for.

Inline emphasis

Use `<strong>` for content of strong importance and `<em>` for emphasis that changes the meaning of a sentence. The older `<b>` and `<i>` exist for stylistic bold and italic with no semantic weight. Most of the time, `<strong>` and `<em>` are what you want.

Inline code and quotations

Use `<code>` for inline code snippets, `<pre>` for preformatted blocks that preserve whitespace, `<blockquote>` for long quotations, and `<q>` for short inline ones. Combine `<pre><code>...</code></pre>` for a multiline code block, which is the convention most CSS libraries expect.

Links and Navigation

The hyperlink is the defining invention of the web. It is also one tag, the anchor: `<a>`. Despite its simplicity, getting links right involves a surprising number of small decisions.

The href attribute

Every meaningful link has an href that specifies where it goes. It can be an absolute URL, a relative path, an anchor on the same page (`#section-id`), or a special scheme such as `mailto:` or `tel:`.

```
<a href="https://example.com">External site</a>
<a href="/about">About this site</a>
<a href="#chapter-3">Jump to Chapter 3</a>
<a href="mailto:hi@example.com">Email me</a>
```

Opening in a new tab

Add `target="_blank"` to open the link in a new browser tab. When you do, also add `rel="noopener noreferrer"` for security: it prevents the new page from gaining a reference back to your window via `window.opener`, a classic phishing vector.

Link text matters

Avoid "click here" or "read more" as link text. Screen readers often present a list of links out of context; "click here" tells the user nothing. Write link text that describes the destination, such as "the 2024 accessibility report".

Navigation

Wrap your primary site navigation in `<nav>`. Inside, an unordered list of links is the convention. Screen readers expose `<nav>` as a landmark, letting users jump straight to it. A page may contain multiple `<nav>` elements as long as they are clearly labelled.

Lists: Ordered, Unordered, and Definition

Lists communicate that items belong together. HTML offers three kinds, each with its own semantics: ordered, unordered, and description (sometimes called definition) lists.

Unordered lists

Use `<ul>` when the order of items does not matter. Each item is a `<li>`. Common uses include navigation menus, tag lists, and groups of feature bullets.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Ordered lists

Use `<ol>` when sequence matters: recipe steps, tournament rankings, version history. The browser numbers items automatically. You can control the starting number with the `start` attribute and reverse the order with `reversed`.

Description lists

Use `<dl>` for name/value pairs such as a glossary or metadata. The `<dt>` is the term and `<dd>` is its description. One term can have many descriptions and vice versa.

```
<dl>
  <dt>HTML</dt>
  <dd>The structure of a page.</dd>
  <dt>CSS</dt>
  <dd>The presentation of a page.</dd>
</dl>
```

Nesting lists

Lists can be nested by placing another `<ul>` or `<ol>` inside a `<li>`. Browsers indent nested lists automatically and switch the bullet or numbering style for each level.

Images and Multimedia

Images, audio, and video are first-class citizens in HTML, but they come with responsibilities: alternative text for accessibility, responsive sources for performance, and captions for legal and usability reasons.

The `<img>` element

Two attributes are required for every image: `src` points to the file, and `alt` describes the image for users who cannot see it. If an image is purely decorative, set `alt=""` so screen readers skip it. Never omit `alt` entirely.

```

```

Responsive images

Use the `srcset` attribute to provide multiple resolutions and let the browser pick the best one for the device. Combine with `sizes` to tell the browser how wide the image will be rendered. This saves bandwidth on small screens.

Audio and video

The `<audio>` and `<video>` elements work similarly: a `src` attribute or one or more `<source>` children, plus the `controls` attribute to show the default playback UI. Always include a fallback caption track (`<track kind="captions">`) for accessibility.

The `<picture>` element

When you need true art direction (different crops or formats per device), use `<picture>`. Each `<source>` can match a media query or a file type, and the browser picks the first matching one. The `<img>` inside is the universal fallback.

Tables

Tables are for tabular data: spreadsheets, schedules, comparison matrices. They are not for page layout. Using tables to position elements is a long-dead technique that ruins accessibility and responsiveness.

Anatomy of a table

A table is a `<table>` containing rows (`<tr>`). Each row contains header cells (`<th>`) or data cells (`<td>`). The optional `<thead>`, `<tbody>`, and `<tfoot>` elements group rows semantically and make long tables easier to scroll.

```
<table>
  <thead>
    <tr><th>Language</th><th>Year</th></tr>
  </thead>
  <tbody>
    <tr><td>HTML</td><td>1993</td></tr>
    <tr><td>CSS</td><td>1996</td></tr>
  </tbody>
</table>
```

Captions and scope

A `<caption>` right after the opening `<table>` describes the whole table for screen readers. Use `scope="col"` or `scope="row"` on `<th>` cells so assistive technology can correctly associate header cells with their data cells. This is essential for complex tables.

Spanning cells

The `colspan` and `rowspan` attributes let a single cell occupy multiple columns or rows. Use sparingly. Complex spanning patterns are hard for screen readers to navigate; consider splitting the data into multiple simpler tables instead.

Forms and Input Types

Forms are how users send data back to your server. Even on a modern single-page app, the underlying mental model is still the HTML form. Getting it right buys you accessibility, autofill, and keyboard navigation for free.

Form structure

A `<form>` has an action (where to send the data) and a method (GET or POST). Inside, each control should be wrapped in a `<label>`. Labels are not optional: they are the difference between a usable form and an inaccessible one.

```
<form action="/signup" method="post">
  <label>
    Email
    <input type="email" name="email" required>
  </label>
  <button type="submit">Sign up</button>
</form>
```

Input types

The type attribute on `<input>` unlocks built-in validation and the right mobile keyboard. Use email, url, tel, number, date, time, color, range, and file when they apply. Browsers handle parsing, validation, and UI hints for you.

Other controls

`<textarea>` is for multi-line text. `<select>` with `<option>` children is for dropdowns. `<input type="checkbox">` and `<input type="radio">` handle multiple-choice answers. Group related radio buttons by giving them the same name attribute.

Client-side validation

Attributes such as `required`, `minlength`, `maxlength`, `min`, `max`, and `pattern` give you free client-side validation. Always validate again on the server, since users can disable or bypass anything in the browser.

Semantic HTML

Semantic HTML means using tags that describe the meaning of content, not its appearance. A page built from `<div>` and `<span>` still works visually, but accessibility tools, search engines, and future maintainers all lose their map of what is what.

Page landmarks

Use `<header>` for the introductory area of the page or a section, `<nav>` for primary navigation, `<main>` for the unique main content of the page, `<aside>` for tangential content like sidebars, and `<footer>` for the closing area. There should be exactly one `<main>` per page.

Content sections

Use `<section>` for thematic groupings of content with their own heading, `<article>` for self-contained pieces that could stand alone (blog posts, news stories), and `<figure>` with `<figcaption>` for images, diagrams, or code samples that need a caption.

Why bother

Screen readers expose landmarks as a navigation menu, so a blind user can jump from header to main to footer instantly. Search engines weigh content inside `<main>` and `<article>` more heavily. And six months from now, your future self reading the source will thank you.

A worked example

```
<body>
  <header><h1>My Blog</h1></header>
  <nav>...</nav>
  <main>
    <article>
      <h2>Post title</h2>
      <p>...</p>
    </article>
  </main>
  <footer>(c) 2026</footer>
</body>
```

HTML Metadata and the Head

The `<head>` is invisible to the user, but it controls how the browser, search engines, and social media platforms interpret your page. A few small tags here have outsized impact.

Character encoding

Always include `<meta charset="utf-8">` as early as possible in the head. Without it, accented characters and emoji can render as garbled mojibake. UTF-8 is the universal default and should never be changed.

The viewport meta tag

For your site to render correctly on phones, you must include `<meta name="viewport" content="width=device-width, initial-scale=1">`. Without it, mobile browsers pretend the screen is 980px wide and zoom out, ruining responsive layouts.

Title and description

`<title>` sets the browser tab and the default search-result link text. `<meta name="description">` sets the snippet shown under search results. Both should be unique per page and under 160 characters.

Open Graph and Twitter Cards

Social platforms read Open Graph (og:) and Twitter Card (twitter:) meta tags to generate link previews. Set `og:title`, `og:description`, `og:image`, and `og:url` at a minimum. A good preview image roughly doubles click-through rate on shared links.

```
<meta property="og:title" content="My Page">
<meta property="og:description" content="Short summary">
<meta property="og:image" content="/preview.png">
```


Accessibility Basics (a11y)

Accessibility is the practice of building sites that work for everyone, including users with disabilities. It is not a checklist; it is a quality bar. Most of the work is just writing HTML correctly, which is why it appears in the HTML half of this book.

The big wins

- Use semantic HTML, not `<div>` soup.
- Label every form control.
- Write meaningful alt text for every image.
- Maintain sufficient color contrast (4.5:1 for body text).
- Make every interaction reachable by keyboard.
- Never disable focus outlines without providing a replacement.

ARIA

ARIA (Accessible Rich Internet Applications) attributes let you add accessibility information that HTML cannot express on its own. But ARIA is a sharp tool: incorrect ARIA is worse than no ARIA. The first rule of ARIA is to use a native HTML element instead, whenever possible.

Testing

Run a tool such as axe DevTools or Lighthouse on every page. Then unplug your mouse and try to use the site with only the keyboard. Finally, turn on VoiceOver on macOS or NVDA on Windows and listen to your own page. You will find problems no automated tool catches.

It is the law

In many countries, public-facing sites are legally required to meet accessibility standards such as WCAG 2.1 AA. Beyond the ethics, building accessible sites is also good defensive practice for your employer.

Part II

CSS: The Look and Feel

Introduction to CSS

CSS, short for Cascading Style Sheets, describes how HTML elements should be presented. It controls colors, sizes, positions, animations, and the responsive behavior of every element on the page. Without CSS, every page would look like a 1993 academic paper.

A rule

A CSS rule consists of a selector, which picks elements, and a block of declarations between curly braces. Each declaration is a property and a value separated by a colon and terminated by a semicolon.

```
p {  
  color: #222;  
  font-size: 16px;  
  line-height: 1.5;  
}
```

Where CSS lives

There are three places to put CSS: in an external file linked from `<head>`, in a `<style>` block in the head, or inline on a single element via the `style` attribute. Always prefer external stylesheets. They are cacheable, reusable, and separate concerns.

The cascade

If two rules target the same element with the same property, the cascade decides which wins. The order of precedence is, roughly: more specific selectors beat less specific ones, and within the same specificity, later rules override earlier ones. Inline styles win over external stylesheets except when `!important` is involved (avoid `!important`).

CSS Selectors and Specificity

Selectors are how you target elements. CSS gives you a rich language for picking exactly the elements you want, from broad (all paragraphs) to narrow (the third item in a specific list inside a specific section).

The core selectors

- `*` matches every element.
- `p` matches every `<p>`.
- `.btn` matches every element with `class="btn"`.
- `#hero` matches the element with `id="hero"`.
- `a[href^="https"]` matches links whose href starts with https.

Combinators

Combinators describe relationships between elements. A space means descendant: `"nav a"` matches any `<a>` inside a `<nav>`. A greater-than sign means direct child: `"nav > a"`. A plus means adjacent sibling, and a tilde means general sibling.

Specificity

Specificity is a four-part score: inline styles, IDs, classes and attribute selectors, and element selectors. Compare from left to right. (1,0,0,0) beats (0,9,9,9). When in doubt, count: one ID is worth one hundred classes.

Practical advice

Keep selectors short. Long chains such as `"body div.container section.posts article.post h2.title"` are brittle and hard to override. Most teams settle on a flat naming convention like BEM (block__element--modifier) to keep specificity uniform.

The Box Model

Every element on a page is, geometrically, a box. The box has four layers: the content itself, padding around it, a border around that, and a margin outside the border. Understanding this model is the difference between confident layout work and endless trial and error.

The four layers

- content: the actual text or image inside the element.
- padding: space between the content and the border.
- border: a line drawn around the padding.
- margin: transparent space outside the border.

Width and height

By default, the width and height properties apply only to the content area. So an element with width: 200px and padding: 20px actually takes up 240px of horizontal space. This catches every new developer at least once.

box-sizing: border-box

Almost every team sets box-sizing: border-box globally. With this, width and height include padding and border, which is what you usually want. The classic reset is one line at the top of your stylesheet:

```
*, *::before, *::after { box-sizing: border-box; }
```

Margin collapse

Vertical margins between adjacent block elements collapse into the larger of the two, not the sum. Horizontal margins never collapse, and neither do margins on flex or grid items. This asymmetry is surprising the first time you hit it.

Display, Position, and Float

Before flexbox and grid existed, layout in CSS was a careful dance of display, position, and float. Modern code uses flex and grid for most layouts, but understanding these older tools is still essential because they explain how the rest of CSS behaves.

Display

The display property controls how an element participates in layout. block elements stack vertically and take full width. inline elements flow horizontally inside text and ignore width and height. inline-block is the hybrid: it sits in text flow but respects width and height. none removes the element from the layout entirely.

Position

static is the default; the element sits where it would normally. relative shifts the element from its normal spot without affecting other elements. absolute removes it from normal flow and positions it relative to the nearest positioned ancestor. fixed positions it relative to the viewport. sticky behaves like relative until you scroll past a threshold, then like fixed.

```
.sticky-header {  
  position: sticky;  
  top: 0;  
  background: white;  
}
```

Float

Float was originally intended for wrapping text around images, and that is still its best use. For years it was abused for multi-column layouts, but flex and grid now do that job far better. Reach for float only when you genuinely need text to flow around an element.

z-index

z-index controls stacking order on the z-axis, but only on elements that are positioned (anything but static). Higher numbers stack on top. Stacking contexts are a subtle topic; when an element creates a new context, its z-index only competes within that context.

Flexbox: The Modern Layout Tool

Flexbox solves one-dimensional layout: arranging items in a row or a column with control over spacing, alignment, and wrapping. If you remember nothing else, remember that flex is for one axis at a time. Grid is for two.

The flex container

Set `display: flex` on a parent and its direct children become flex items. By default they line up in a row. The container controls the axes; the items control how they grow and shrink to fill them.

```
.bar {  
  display: flex;  
  gap: 1rem;  
  justify-content: space-between;  
  align-items: center;  
}
```

Main and cross axis

The main axis is the direction items flow (row by default). The cross axis is perpendicular. `justify-content` aligns items along the main axis; `align-items` aligns them along the cross axis. Memorize that and half of flexbox becomes intuitive.

Growing and shrinking

`flex: 1` on an item lets it grow to fill remaining space. `flex: 0 0 200px` sets a fixed width. The shorthand is `flex: <grow> <shrink> <basis>`. The grow value is by far the most useful in practice.

Wrapping

By default, flex items shrink to fit on one line. Set `flex-wrap: wrap` to allow them to spill onto multiple lines. Combined with a `min-width` on items, this is a quick way to build responsive card grids without media queries.

CSS Grid: Two-Dimensional Layouts

Grid is the most powerful layout system CSS has ever had. It controls rows and columns at the same time, lets you name regions, and removes the need for layout libraries. The cost is that it has more concepts than flexbox.

Defining a grid

Set `display: grid` on a parent and then declare the rows and columns. The `fr` unit means "share of the remaining space", so `1fr 2fr 1fr` makes three columns where the middle one is twice as wide as the others.

```
.layout {  
  display: grid;  
  grid-template-columns: 200px 1fr 200px;  
  grid-template-rows: auto 1fr auto;  
  gap: 1rem;  
  min-height: 100vh;  
}
```

Placing items

Items flow into the grid in source order, filling cells one by one. You can also place an item explicitly by line numbers or by naming areas. Named areas are especially readable for page layouts.

```
.layout {  
  grid-template-areas:  
    "header header header"  
    "nav    main    aside"  
    "footer footer footer";  
}  
.main { grid-area: main; }
```

Auto-placement and minmax

`repeat(auto-fill, minmax(200px, 1fr))` is the magic incantation for a responsive card grid: as many columns as fit, each at least 200px, sharing remaining space equally. No media queries required.

Typography in CSS

Typography is the most consequential decision on most websites. Good type makes everything else feel more polished, and CSS gives you precise control over every aspect of it.

Font family

Set fonts via the `font-family` property. Always include a list of fallbacks, ending in a generic family such as `sans-serif` or `serif`. The browser tries each name in order until it finds one the user has installed.

```
body {  
  font-family: "Inter", system-ui, -apple-system, sans-serif;  
}
```

Size, weight, and line height

Use `rem` for font-size in most cases: `1rem` equals the root font size, so users who change their browser default get a consistent scale. Set `line-height` as a unitless number (e.g. `1.5`), which scales correctly with the font size.

Web fonts

Self-host fonts using `@font-face` with `font-display: swap`, which shows fallback text immediately and swaps in the custom font when it loads. This prevents the dreaded flash of invisible text. Subset large fonts to only the characters you need.

Rhythm

Pick a small set of font sizes and spacing values and reuse them everywhere. A type scale such as 12, 14, 16, 20, 24, 32, 48 covers almost every use case and keeps your interface visually consistent without conscious effort.

Colors, Gradients, and Backgrounds

CSS has more ways to express colors than any other language, and background features that turn the language into a mini-graphics engine. Used well, they replace dozens of image assets.

Color formats

Use hex (`#1a2b3c`) for opaque colors and `rgb()`/`hsl()` when you need transparency or a more intuitive way to derive variants. Modern browsers also support `oklch()`, which gives perceptually uniform color manipulation, a huge win for designing themes.

Gradients

`linear-gradient()` and `radial-gradient()` create gradients that behave like images. They can be stacked, layered, and animated. `conic-gradient()` is the third type and is great for pie-chart-like visuals.

```
.hero {  
  background: linear-gradient(135deg, #4f46e5, #ec4899);  
}
```

Background shorthand

`background` is a shorthand that sets image, position, size, repeat, attachment, and color in one declaration. Knowing the individual properties helps you read other people's code, but the shorthand is usually more compact for your own.

Multiple backgrounds

You can stack multiple background images on a single element, separated by commas. The first one in the list is on top. Combine a gradient over an image for tasteful overlays without an extra `<div>`.

Transitions and Animations

Motion done well makes an interface feel alive and responsive. Motion done badly is annoying or, worse, harmful to users with vestibular sensitivities. CSS gives you two tools: transitions and keyframe animations.

Transitions

A transition smoothly interpolates between two property values when the value changes (for example, on hover). Specify which properties to animate, the duration, and an easing function.

```
.btn {  
  background: #4f46e5;  
  transition: background 200ms ease;  
}  
.btn:hover { background: #4338ca; }
```

Keyframe animations

For multi-step or repeating motion, define a `@keyframes` block and apply it with the `animation` property. Keyframes can use percentages from 0% to 100% to describe intermediate states.

```
@keyframes pulse {  
  0%, 100% { opacity: 1; }  
  50%      { opacity: 0.4; }  
}  
.loading { animation: pulse 1.5s ease-in-out infinite; }
```

Respect user preferences

Some users have requested less motion at the OS level. Honor it with a media query: `@media (prefers-reduced-motion: reduce) { * { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; } }`.

Responsive Design and Media Queries

Responsive design is the practice of making one site work on every screen, from phones to ultrawide monitors. It starts with a fluid layout and adds media queries only where necessary.

Mobile-first

Write the small-screen styles first, then layer larger-screen overrides with min-width media queries. This produces less code and degrades better when something goes wrong. Designing in the opposite direction tends to bolt on "mobile fixes" that grow messy fast.

Media query syntax

```
@media (min-width: 768px) {  
  .layout { grid-template-columns: 1fr 2fr; }  
}
```

Common breakpoints

There is no universal breakpoint set. Common values are 640px, 768px, 1024px, and 1280px, but you should choose breakpoints where your specific layout breaks, not where the framework says they should be.

Fluid units

Prefer percentage, em, rem, ch, fr, and viewport units over fixed pixels. `clamp(min, ideal, max)` is a single-function responsive primitive: `clamp(1rem, 2vw + 1rem, 2rem)` gives you fluid typography that never gets absurdly small or large.

Container queries

Container queries respond to the size of a parent element rather than the viewport. This is exactly what component libraries always wanted, and it is now broadly supported. Use them when you ship a component that is meant to be reused at arbitrary widths.

CSS Variables (Custom Properties)

CSS variables let you store values once and reuse them anywhere. Unlike preprocessor variables, they are live: you can change them at runtime with JavaScript, override them in media queries, or set different values per-component.

Defining and using

Declare a variable with a name starting with two dashes, typically on the `:root` pseudo-class so it is available everywhere. Use it with the `var()` function.

```
:root {
  --color-brand: #4f46e5;
  --radius: 8px;
}
.btn {
  background: var(--color-brand);
  border-radius: var(--radius);
}
```

Theming

The classic use of variables is theming. Define a light theme on `:root` and override the same variables under a `[data-theme="dark"]` selector. Every component referencing `var(--color-bg)` updates automatically. No JavaScript class-swap gymnastics needed.

Fallbacks

`var(--name, fallback)` lets you provide a default if the variable is not defined. Useful when shipping a component that should work even if the consumer has not set up your tokens.

JavaScript interop

Read variables with `getComputedStyle(el).getPropertyValue("--x")` and write them with `el.style.setProperty("--x", value)`. This is how modern frontends drive transitions and animations from scripted logic without rewriting style rules.

Pseudo-classes and Pseudo-elements

Pseudo-classes target an element based on its state or position. Pseudo-elements target a sub-part of an element. Together they let you describe rich interactions in pure CSS, often replacing JavaScript entirely.

State pseudo-classes

- `:hover` when the pointer is over the element.
- `:focus` when the element has keyboard focus.
- `:focus-visible` only when the focus came from the keyboard.
- `:active` during the moment of clicking or tapping.
- `:disabled` when a form control is disabled.

Structural pseudo-classes

`:first-child`, `:last-child`, `:nth-child(n)`, and `:not(selector)` let you target elements by position or by what they are not. `:nth-child` accepts formulas: $2n$ picks every even child, $3n+1$ picks every third starting from the first.

Pseudo-elements

`::before` and `::after` generate content immediately inside the selected element, before or after its real children. They are perfect for decorative icons, quotation marks, or visual flair that has no semantic meaning. `::placeholder` styles the placeholder text inside form fields.

```
.quote::before { content: "\201C"; font-size: 2em; }
```

Why bother

Done well, these features remove the need for entire categories of DOM elements and JavaScript handlers. A tooltip can be pure CSS with `:hover` and a pseudo-element. A custom checkbox can be pure CSS with `:checked` and `::before`.

Part III

JavaScript: The Behavior Layer

Introduction to JavaScript

JavaScript is the language that makes web pages interactive. Originally a quick prototype written in ten days, it now powers everything from button click handlers to entire backend servers, mobile apps, and desktop applications.

Where it runs

In a browser, JavaScript runs inside a sandboxed engine such as V8 (Chrome, Edge), SpiderMonkey (Firefox), or JavaScriptCore (Safari). On the server it runs under Node.js, Deno, or Bun. The core language is the same; the available APIs differ.

Adding it to a page

Use `<script src="app.js"></script>`, ideally with the `defer` attribute so it does not block parsing of the HTML. Place script tags at the end of `<body>` or in `<head>` with `defer`. Inline `<script>` blocks work too but are harder to cache and reuse.

```
<script src="app.js" defer></script>
```

Modules

Modern code uses ES modules. Add `type="module"` to the script tag, then use `import` and `export` to split your code across files. Modules are deferred by default and run in strict mode automatically.

```
<script src="app.js" type="module"></script>
```

Your first script

`console.log("hello")` in your script will print to the browser DevTools console. Open the console (Cmd+Option+J on Chrome) and you can type expressions live. The console is the single most useful tool for learning the language.

Variables, Data Types, and Operators

JavaScript variables come in three flavors and have a small set of primitive types. Mastering the distinctions early prevents a lot of confusion later.

let, const, and var

Use `const` by default and reach for `let` only when you need to reassign the variable. Avoid `var`; it has loose scoping rules that lead to bugs. The choice has nothing to do with whether the underlying value is mutable; `const` just prevents reassignment of the binding.

```
const PI = 3.14159;  
let count = 0;  
count = count + 1;
```

Primitive types

There are seven: string, number, boolean, null, undefined, bigint, and symbol. Everything else (arrays, functions, dates) is an object. `typeof null` returns "object", which is a well-known historical quirk.

Type coercion

JavaScript happily converts between types in surprising ways: `"5" + 1` is `"51"` but `"5" - 1` is `4`. Use `===` instead of `==` for equality to avoid silent coercion. Most codebases lint against `==` entirely.

Operators worth knowing

- `??` returns the right side only if the left is null or undefined.
- `?.` safely chains property access: `user?.address?.city`.
- `**` is exponentiation: `2 ** 10` is 1024.
- `+=` `-=` `*=` `/=` are compound assignment shortcuts.
- Spread `...` copies into arrays and objects.

Control Flow: Conditionals and Loops

Control flow is how a program decides what to do next. JavaScript has the standard set of conditionals and loops, plus a few modern shortcuts worth knowing.

If/else

if and else are exactly what you'd expect. else if chains long conditions, but if you find yourself writing more than three branches, a switch or a lookup object is usually clearer.

```
if (score >= 90) { grade = "A"; }  
else if (score >= 80) { grade = "B"; }  
else { grade = "C"; }
```

Ternary

condition ? then : else is a compact way to write a small branch as an expression. It is perfect inside template strings and JSX. Avoid nesting ternaries; they get unreadable fast.

Loops

for, while, and do-while exist. In practice you rarely write them: for-of iterates values of arrays or other iterables, for-in iterates keys of objects (use sparingly), and array methods such as map, filter, and forEach replace most explicit loops.

```
for (const name of names) {  
  console.log(name);  
}
```

break and continue

break exits a loop early; continue skips to the next iteration. Inside switch, break exits the switch. Forgetting break in a switch causes fall-through to the next case, which is rarely what you want.

Functions and Scope

Functions are the fundamental unit of reuse in JavaScript. The language has multiple ways to declare them, each with subtly different behavior.

Declarations vs expressions

```
function greet(name) { return "hi " + name; }
const greet2 = function(name) { return "hi " + name; };
const greet3 = (name) => "hi " + name;
```

Arrow functions

Arrow functions are shorter, cannot be used as constructors, and do not have their own this. The lack of this binding is their main feature: inside a method, an arrow inherits the outer this, which removes a whole category of bugs.

Default and rest parameters

Set defaults right in the parameter list: `function f(x = 1) {}`. Use `...rest` to collect remaining arguments into an array: `function f(first, ...rest) {}`. These features replace older, more error-prone patterns.

Scope and closures

Each function creates a new scope. Inner functions can read outer variables, even after the outer function has returned. This is called a closure and is the foundation of every JavaScript pattern from event handlers to module systems.

```
function counter() {
  let n = 0;
  return () => ++n;
}
const next = counter();
next(); // 1
next(); // 2
```

Arrays and Objects

Arrays and objects are the only two collection types in JavaScript, and almost every data structure is built from them. Their methods are where most of the language's expressive power lives.

Arrays

Arrays are ordered lists with integer indices. Create them with literal syntax: `const xs = [1, 2, 3]`. `push` and `pop` work at the end, `shift` and `unshift` at the start. `length` is the count.

Functional array methods

`map`, `filter`, `reduce`, `find`, `some`, `every`, and `flatMap` each take a callback and return a new array (or value). They do not mutate the original array, which makes them safe to use anywhere.

```
const xs = [1, 2, 3, 4];
const evens = xs.filter(n => n % 2 === 0);
const doubled = xs.map(n => n * 2);
const sum = xs.reduce((acc, n) => acc + n, 0);
```

Objects

Objects are unordered collections of key/value pairs. Keys are strings (or symbols), values are anything. Create with literal syntax: `const user = { name: "Ana", age: 30 }`. Access with `user.name` or `user["name"]`.

Destructuring and spread

`const { name, age } = user` lets you pull properties out by name. `const [first, ...rest] = arr` does the same for arrays. The spread operator builds new arrays and objects: `const copy = { ...user, age: 31 }` creates a copy with `age` overridden.

The DOM: Selecting and Modifying Elements

The Document Object Model is the bridge between HTML and JavaScript. It exposes every element on the page as an object you can read and modify. Modifying the DOM is how a script changes what the user sees.

Selecting elements

`querySelector` returns the first element matching a CSS selector; `querySelectorAll` returns all of them as a `NodeList`. Older `getElementById` and `getElementsByClassName` still work, but `querySelector` is more general.

```
const btn = document.querySelector("#submit");
const items = document.querySelectorAll(".item");
```

Reading and writing content

`textContent` reads or sets the text inside an element. `innerHTML` does the same with HTML, but watch out: setting `innerHTML` with user-supplied data is the classic cross-site scripting vulnerability. Prefer `textContent` unless you genuinely need HTML.

Attributes and properties

`el.getAttribute` and `el.setAttribute` read and write attributes. For many common attributes there is a more convenient property: `el.id`, `el.className`, `el.value`, `el.checked`. Use the property when it exists.

Class and style manipulation

`el.classList.add`, `remove`, `toggle`, and `contains` manage classes. `el.style.color = "red"` sets inline styles, which is fine for dynamic values but generally less maintainable than toggling a class that has the style defined in CSS.

Creating and inserting

`document.createElement("li")` makes a new element. `parent.appendChild(child)` puts it in the DOM. `parent.append` accepts multiple children at once and even raw strings.

Events and Event Handling

Events are how the browser tells your code that something has happened: a click, a keystroke, a network response, a page load. Handling them well is the core of every interactive feature.

addEventListener

Attach a handler with `el.addEventListener("click", handler)`. Remove it with `removeEventListener`. The older `onclick` property works too but only allows one handler at a time, which is rarely what you want.

```
btn.addEventListener("click", (event) => {  
  console.log("clicked", event.target);  
});
```

The event object

Handlers receive an event object with properties such as `target` (the element the event fired on), `currentTarget` (the element the handler is attached to), and methods like `preventDefault` and `stopPropagation`.

Bubbling and delegation

Most events bubble up from the element they originate on through every ancestor. This lets you attach one handler to a parent and respond to events from any child: event delegation. It is essential for dynamic lists where children come and go.

```
list.addEventListener("click", (event) => {  
  const li = event.target.closest("li");  
  if (li) handle(li);  
});
```

Removing and aborting

Pass a function reference, not an anonymous arrow, if you ever want to remove the handler later. Or use an `AbortController` to remove many handlers at once: pass `{ signal }` and later call `controller.abort()`.

Asynchronous JavaScript

JavaScript is single-threaded, which means it can only do one thing at a time. Asynchronous code is how the language stays responsive: it lets long operations such as network requests complete in the background while the main thread keeps running.

Callbacks

The original async pattern: pass a function that runs when work completes. `setTimeout(() => {...}, 1000)` is a callback. Old APIs nest callbacks inside callbacks, leading to the famous "callback hell".

Promises

A Promise represents a value that will exist eventually. It can be pending, fulfilled, or rejected. Chain handlers with `then` and `catch`. The chain replaces nested callbacks with a flat sequence of steps.

```
fetch("/api/users")
  .then(res => res.json())
  .then(users => render(users))
  .catch(err => console.error(err));
```

async/await

`async` and `await` are sugar on top of promises that let you write asynchronous code that reads like synchronous code. An `async` function always returns a promise; `await` pauses inside it until a promise resolves.

```
async function loadUsers() {
  try {
    const res = await fetch("/api/users");
    const users = await res.json();
    render(users);
  } catch (err) {
    console.error(err);
  }
}
```

Parallel work

`Promise.all` runs an array of promises in parallel and waits for all of them. `Promise.race` resolves with whichever finishes first. `Promise.allSettled` waits for everything to either resolve or reject, then gives you the full report.

ES6+ Features Worth Knowing

Modern JavaScript has grown a number of features that make everyday code shorter and safer. Here are the ones you will actually use, with no nostalgia for the older equivalents.

Template literals

Backtick-delimited strings can span multiple lines and embed expressions with `${ ... }`. They replace ugly string concatenation entirely.

```
const greeting = `Hello, ${user.name}!
Welcome back.`;
```

Optional chaining and nullish coalescing

`user?.address?.city` returns undefined instead of throwing if any link in the chain is null. `value ?? "fallback"` returns the fallback only if value is null or undefined, leaving empty strings and zeros intact.

Object shorthand and computed keys

`{ name, age }` is shorthand for `{ name: name, age: age }`. `{ [key]: value }` lets you use a runtime expression as a property name. These two features make object literals dramatically less verbose.

Modules

Modern code is organized into modules with `import` and `export`. Each module is its own scope. Default exports are convenient for the primary thing a file exports; named exports are better for utility modules with several pieces.

```
// utils.js
export function add(a, b) { return a + b; }
// main.js
import { add } from "./utils.js";
```

Classes

`class` is syntactic sugar over prototypes. You get clean syntax for constructors, methods, static methods, getters, setters, and inheritance with `extends`. Use them when modeling a clear domain object; reach for plain objects when there is no behavior.

Part IV

Beyond the Basics

Embedded Content: iframe, object, embed

Sometimes you need to embed a page inside another page: an interactive demo, a third-party widget, a sandboxed preview. HTML gives you a handful of elements for the job, each with subtly different rules.

The <iframe> element

An iframe loads an entire HTML document inside the current one. It has its own window, its own scripts, and (usually) its own security context. The src attribute is the URL to load. width and height control the box size.

```
<iframe src="https://example.com/widget"
        width="600" height="400"
        title="Example widget"
        loading="lazy"></iframe>
```

Sandboxing

The sandbox attribute drops every permission the iframe would normally have: scripts, forms, navigation, popups. Add back only what you need: sandbox="allow-scripts allow-forms". This is essential when embedding user-supplied content.

Same-origin and CORS

If the iframe is on the same origin as the parent, both can read and write each other's DOM. If not, browsers block all cross-document access for security. Use postMessage for safe, explicit communication across origins.

Lazy loading

Add loading="lazy" to iframes (and images) that are below the fold. The browser defers loading them until they are about to scroll into view, saving bandwidth and speeding up the initial page load.

Canvas, SVG, and Drawing

HTML offers two complementary ways to draw graphics in the browser: SVG, which is declarative and resolution-independent, and Canvas, which is procedural and pixel-based. The choice depends on what you are drawing.

When to use SVG

SVG is best for icons, diagrams, and any image with a manageable number of distinct shapes. Each shape is a DOM element you can style with CSS, animate, or query with JavaScript. SVGs scale to any size without losing quality.

When to use Canvas

Canvas is best for high-frequency drawing: charts with thousands of points, games, image filters. You get a single bitmap that you redraw imperatively. Performance scales much further than SVG, at the cost of losing DOM-based interaction.

```
const canvas = document.querySelector("canvas");
const ctx = canvas.getContext("2d");
ctx.fillStyle = "#4f46e5";
ctx.fillRect(10, 10, 100, 50);
```

Inline SVG vs external SVG

Inline SVG (a `<svg>` element directly in HTML) gives you full CSS and JavaScript control over every shape. External SVG (loaded via `<img>`) caches better but is opaque to CSS. Most icon systems inline their SVG to allow color theming.

Accessibility

An `<svg>` used as an icon should have `role="img"` and a `<title>` child with descriptive text. A purely decorative SVG should have `aria-hidden="true"` so screen readers skip it. Canvas is invisible to assistive tech; provide a text alternative.

Microdata, Schema.org, and SEO

Search engines and social platforms try hard to understand what a page is about. You can help them by annotating your HTML with structured data. Done right, this unlocks rich search results: star ratings, recipe cards, event dates.

JSON-LD

The preferred format today is JSON-LD: a `<script type="application/ld+json">` block in the page head. It uses vocabulary from schema.org, a shared vocabulary maintained by Google, Bing, Yahoo, and Yandex.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "My Post",
  "author": { "@type": "Person", "name": "Ana" },
  "datePublished": "2026-05-18"
}
</script>
```

Picking the right type

Browse the schema.org vocabulary at schema.org/docs/schemas. Common types include Article, Product, Recipe, Event, FAQPage, Organization, Person, and BreadcrumbList. Each has required and recommended properties documented in detail.

Testing

Use Google's Rich Results Test to validate your structured data before deploying. It catches missing required fields and shows a preview of how the rich result would look in search.

Beyond structured data

Don't forget the basics: a clear `<title>`, a unique meta description, semantic headings, descriptive alt text, and a `sitemap.xml`. No amount of structured data will save a page that is slow, inaccessible, or built without semantic HTML.

CSS Architecture: BEM, ITCSS, and Utility-First

As soon as a project grows beyond a single page, CSS without structure becomes a maintenance nightmare. Three patterns have proven themselves at scale: BEM, ITCSS, and utility-first. Each makes different trade-offs.

BEM

BEM stands for Block, Element, Modifier. Every class is named `block__element--modifier`. The result is verbose but flat: every selector has the same specificity, so the cascade never fights you. Pair with custom properties for theming.

```
.card { ... }  
.card__title { ... }  
.card__image { ... }  
.card--featured { ... }
```

ITCSS

ITCSS (Inverted Triangle CSS) is a layering convention: settings, tools, generic resets, elements, objects, components, utilities. Lower layers are general; higher layers are specific. Importing in this order prevents specificity wars.

Utility-first

Utility-first frameworks like Tailwind ship one tiny class per style: `.p-4` for padding 1rem, `.text-lg` for large text. You compose UI directly in HTML. The result is verbose markup but almost zero custom CSS, and trivial to refactor.

Picking one

For a small site, anything works. For a team-built design system, pick BEM (great for hand-rolled components) or Tailwind (great for application UIs that change often). ITCSS is less of a system and more of a layering discipline you can use alongside either.

CSS Layers, Nesting, and Modern Features

CSS has evolved fast in the last few years. Several long-requested features finally landed across all major browsers. Knowing them changes how you structure stylesheets.

@layer

The @layer rule lets you declare a stack of layers. Rules in a later layer always win over rules in earlier layers, regardless of selector specificity. This finally tames the third-party-CSS specificity problem.

```
@layer reset, base, components, utilities;
@layer base { p { line-height: 1.6; } }
@layer components { .btn { ... } }
```

Native nesting

You can now nest CSS rules natively, the way Sass has done for years. Use it sparingly: deeply nested rules are still hard to read and create high-specificity selectors.

```
.card {
  padding: 1rem;
  & .title { font-weight: bold; }
  &:hover { background: #f0f0f0; }
}
```

:has() the parent selector

:has() finally lets a selector reach back up the tree. li:has(> a.active) selects any list item that contains an active link. This used to require JavaScript.

Subgrid

Grid items can now opt into their parent's grid lines with subgrid. This solves the classic problem of aligning fields across many cards or rows without giving up grid's responsiveness.

Print Stylesheets, Dark Mode, and Media Features

Media queries are not just for screen widths. They expose a rich set of features about how the page is being viewed: device type, color scheme, motion preferences, hover capability, and more. Use them to delight users.

Dark mode

Honor the user's system theme with `@media (prefers-color-scheme: dark)`. Combined with custom properties, you can flip an entire theme in a single block without rewriting selectors.

```
@media (prefers-color-scheme: dark) {  
  :root {  
    --bg: #111;  
    --fg: #eee;  
  }  
}
```

Print styles

`@media print` lets you adapt the page when the user prints or saves to PDF. Hide navigation, expand abbreviations, force page breaks before headings. The result is a vastly nicer printed document for the cost of about 20 lines of CSS.

Hover, pointer, and motion

`(hover: hover)` is true only when the user has a real hover capability, distinguishing desktop from touch. `(pointer: coarse)` detects touch. `(prefers-reduced-motion: reduce)` honors the OS motion setting.

Color gamut and contrast

Newer media features include `color-gamut (sRGB, p3, rec2020)` and `prefers-contrast (more, less)`. Most users will not notice, but the ones who set these preferences notice deeply. Respecting them costs nothing and signals quality.

Strings, Arrays, and Numbers: A Deeper Look

The built-in JavaScript types ship with rich methods that replace whole helper libraries. Knowing them well saves you from inventing slower, buggier versions.

String methods worth memorizing

- `trim()`, `trimStart()`, `trimEnd()` strip whitespace.
- `startsWith(s)`, `endsWith(s)`, `includes(s)` test for substrings.
- `replaceAll(needle, replacement)` replaces every match.
- `split(sep)` returns an array; `join(sep)` is the inverse.
- `padStart(len, char)`, `padEnd(len, char)` align fixed-width text.

Array methods you will use weekly

- `map`, `filter`, `reduce`, `find`, `findIndex`.
- `some`, `every` test if any or all elements pass a predicate.
- `flat()` and `flatMap()` flatten nested arrays.
- `at(-1)` gets the last element without indexing arithmetic.
- `Array.from(iterable)` builds an array from anything iterable.

Number quirks

JavaScript has one number type, a 64-bit float. This means $0.1 + 0.2$ is `0.30000000000000004`. For currency or anything needing exact decimals, multiply to integer cents, or use `BigInt`, or pull in a small decimal library.

Useful number tools

- `Number.isInteger(x)`, `Number.isFinite(x)`.
- `Math.floor`, `Math.round`, `Math.ceil`, `Math.trunc`.
- `Math.max`, `Math.min`, `Math.random`.
- `(0.1 + 0.2).toFixed(2)` formats with fixed decimals.
- `Number("42")` and `parseInt` vs `parseFloat` have different rules.

Error Handling

Things go wrong. Networks drop, JSON is malformed, files are missing. The difference between an annoying app and a robust one is how it handles failures.

try/catch/finally

Wrap code that may throw in try, handle the failure in catch, and run cleanup in finally. The finally block runs whether or not an exception was thrown, even if the try returned early.

```
try {
  const data = JSON.parse(input);
} catch (err) {
  console.error("bad json", err);
} finally {
  hideSpinner();
}
```

Throwing errors

Throw an Error object, not a string. Error objects carry a stack trace that is invaluable when debugging. Use specific subclasses like TypeError, RangeError, or your own custom Error subclasses for domain-specific failures.

Async errors

Inside an async function, a rejected promise becomes a thrown error you can catch with try/catch. Outside async functions, attach a .catch() to the promise chain or you will get the dreaded UnhandledPromiseRejection warning.

Graceful degradation

When an operation fails, show the user something useful: a retry button, a cached version, a clear message. Never just swallow errors silently. Log them to your monitoring system and let the user know what happened.

Working with JSON

JSON is the lingua franca of web APIs. JavaScript has two built-in functions for working with it, and getting them right matters.

JSON.parse and JSON.stringify

JSON.parse(text) turns a JSON string into a JavaScript value. JSON.stringify(value) goes the other way. Both can throw on malformed input, so wrap parse in try/catch.

```
const text = '{"name": "Ana", "age": 30}';
const user = JSON.parse(text);
const back = JSON.stringify(user, null, 2);
```

Pretty printing

The third argument to stringify is the indentation. Pass 2 for two-space indent. The second argument is a replacer function or array, useful for stripping sensitive fields before logging.

What JSON cannot represent

JSON cannot represent functions, undefined, Date objects, Map, Set, BigInt, or circular references. Functions and undefined are silently dropped from objects. Dates become ISO strings (one-way conversion). Plan accordingly.

Streaming and large data

JSON.parse loads the entire string into memory. For large files or streaming APIs (NDJSON, JSON Lines), use a streaming parser that yields one object at a time. The fetch API supports streaming responses for this.

The Fetch API and HTTP

fetch is the modern way to make HTTP requests from JavaScript. It returns a Promise that resolves to a Response object. Mastering it removes any need for older libraries like jQuery or axios in most cases.

A basic request

```
const res = await fetch("/api/users");
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

Why ok matters

fetch only rejects on network errors, not on 4xx or 5xx HTTP responses. Always check res.ok or res.status. Forgetting this is the single most common fetch bug.

Sending data

Use the options object to set method, headers, and body. JSON requires a Content-Type header and a stringified body. FormData is for file uploads and form submissions.

```
await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name }),
});
```

Aborting

Long requests can be cancelled with AbortController. Create one, pass its signal to fetch, and call controller.abort() when the user navigates away or starts a new search. This prevents stale responses from overwriting fresher ones.

CORS

When the browser requests a resource on a different origin, it enforces Cross-Origin Resource Sharing rules. The server must opt in via response headers. If you see a CORS error in DevTools, the fix is on the server, not in your fetch call.

Browser Storage

The browser gives you several ways to store data on the user's machine. Pick the right one for the job: each has different size limits, lifetime rules, and APIs.

localStorage

Synchronous, string-only key-value store. Survives across tabs and browser restarts. Roughly 5MB per origin. Use it for user preferences and small persistent state.

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
localStorage.removeItem("theme");
```

sessionStorage

Same API as localStorage but cleared when the tab closes. Useful for ephemeral form drafts or temporary state that should not leak between sessions.

IndexedDB

An asynchronous, indexed database in the browser. Supports structured data, transactions, and queries. The raw API is verbose; a small wrapper like `idb-keyval` makes it painless for most uses.

Cookies

Cookies are still the only storage automatically sent with each HTTP request, making them necessary for authentication with most server-rendered apps. Set `HttpOnly`, `Secure`, and `SameSite` attributes for any cookie that holds a session.

Cache API

The Cache API stores Response objects, designed to back service workers. Use it to implement offline-first apps that serve cached responses when the network is unavailable.

Security Essentials

Browsers run untrusted code from many origins. The platform is built on layers of defense to keep one site from attacking another. Understanding the threats and the tools to mitigate them is non-negotiable for production code.

Cross-Site Scripting (XSS)

XSS happens when attacker-controlled content is rendered as code in your page. The fix: never insert untrusted strings into the DOM as HTML. Use `textContent`, not `innerHTML`. If you must render HTML, sanitize with a library like `DOMPurify`.

Cross-Site Request Forgery (CSRF)

CSRF tricks an authenticated user's browser into making requests they did not intend. Defenses: SameSite cookies, CSRF tokens on state-changing forms, and using JSON Content-Type which most CSRF attacks cannot forge.

Content Security Policy (CSP)

A CSP header tells the browser exactly which sources are allowed to load scripts, styles, and other resources. A strict CSP eliminates entire classes of XSS by refusing to execute inline scripts or scripts from untrusted origins.

HTTPS everywhere

Always serve every page over HTTPS. The Strict-Transport-Security header tells browsers to refuse plain-HTTP requests to your domain for a year, eliminating downgrade attacks after the first visit.

Don't roll your own crypto

If you find yourself implementing password hashing, encryption, or signature verification by hand, stop. Use established libraries and standards. The SubtleCrypto API is built into every browser for the rare cases where you do need cryptography in JavaScript.

Performance Basics

Performance is a feature. Slow pages lose users, lower search rankings, and drain mobile batteries. A handful of techniques get you most of the way.

Measure first

Use Lighthouse and the WebPageTest service to measure real page performance. The metrics that matter most for users are Largest Contentful Paint, Interaction to Next Paint, and Cumulative Layout Shift, collectively known as Core Web Vitals.

Ship less code

The single biggest win for most sites is shipping less JavaScript. Audit your bundle with a tool like source-map-explorer. Remove unused dependencies. Prefer the platform (dialog, popover, details) over JavaScript replacements.

Images

Images are usually the largest assets on a page. Serve modern formats (AVIF, WebP), include srcset for responsive sizing, lazy-load below the fold, and always specify width and height to prevent layout shift.

Caching

Set far-future cache headers on assets with hashed filenames. The first visit is slow; every subsequent visit and every navigation within the site reuses cached assets. A good CDN turns global latency into local latency.

Avoid layout thrashing

Reading layout properties (offsetTop, getBoundingClientRect) after writing to the DOM forces the browser to recalculate layout synchronously. Batch reads and writes, or use requestAnimationFrame for animation work.

How the Browser Renders a Page

Knowing roughly how a browser turns HTML, CSS, and JavaScript into pixels makes performance work less mysterious. There are five major phases.

Parsing

The browser reads HTML and builds the DOM tree. It reads CSS and builds the CSSOM. `<script>` tags by default block parsing; `defer` and `async` let parsing continue while scripts download in the background.

Style

The browser combines DOM and CSSOM to compute the final style for every visible element. This is where the cascade is resolved. Avoid extremely expensive selectors; the browser must run them against every element.

Layout

Once styles are computed, the browser calculates the position and size of every box on the page. Layout is expensive. Animating properties like `top`, `left`, or `width` triggers layout on every frame, which is why `transform` and `opacity` are the recommended animation properties.

Paint

Each box is rasterized into pixels. Different boxes may land on different layers, especially if they have `will-change` or `transform` applied. Painting is faster than layout but still costs.

Composite

Layers are sent to the GPU and composited into the final image. `Transform` and `opacity` changes can skip layout and paint and go straight to composite, which is why they are the fastest properties to animate.

Forms in JavaScript

Forms remain the workhorse of every web app. Even with modern frameworks, the underlying DOM API is what every abstraction is built on top of. Knowing it directly pays off.

Reading form values

FormData wraps any <form> element and gives you a Map-like interface to its fields. It handles every input type, including file uploads and multi-select. Pass it directly to fetch as a body.

```
form.addEventListener("submit", async (e) => {  
  e.preventDefault();  
  const data = new FormData(form);  
  await fetch("/api/save", { method: "POST", body: data });  
});
```

preventDefault

Without preventDefault, the browser will navigate to the form's action URL on submit. For client-side handling, always call it first inside the submit handler. Otherwise your fetch will race against a page navigation.

Validation API

Each input exposes a validity object describing why it is invalid: valueMissing, typeMismatch, tooShort, patternMismatch. Use setCustomValidity to add domain-specific validation. Calling form.reportValidity() shows the browser's native validation messages.

Controlled vs uncontrolled

In React (and similar frameworks), a controlled input stores its value in state and writes it back on every keystroke. An uncontrolled input leaves the value in the DOM and reads it on submit. Uncontrolled is faster but harder to validate live; controlled is the opposite.

Browser DevTools

DevTools is the most useful, most underused feature of the modern browser. Every panel rewards investment; together they replace half a dozen separate tools you might otherwise install.

Elements

Inspect any element, see its computed style, and edit HTML or CSS live. The styles pane shows which rules apply, with overridden ones struck through. The box model diagram visualizes margins and padding at a glance.

Console

Run any JavaScript expression against the current page. `$0` is the currently selected element in the Elements panel. `$(selector)` is shorthand for `document.querySelectorAll`. `console.table(array)` pretty-prints arrays of objects.

Network

Every HTTP request and response, with headers, timing, and payload. Throttle to slow 3G to test mobile performance. The waterfall view reveals which requests are blocking the page from rendering.

Performance

Record a profile while reproducing a slow interaction. The resulting flame chart shows exactly which functions, layouts, and paints took how long. Most performance problems become obvious after one good profile.

Application

Inspect `localStorage`, `sessionStorage`, cookies, `IndexedDB`, cache storage, and service workers. Clear them all when debugging weird stale-data behavior. The Service Workers section is essential for any offline-capable app.

Putting It All Together

You now have the vocabulary to build a real, modern web page from scratch, without a framework. Here is what a minimal but professional starting point looks like.

The HTML shell

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport"
      content="width=device-width, initial-scale=1">
    <title>My App</title>
    <link rel="stylesheet" href="styles.css">
    <script src="app.js" type="module" defer></script>
  </head>
  <body>
    <main id="root"></main>
  </body>
</html>
```

The CSS foundation

```
*, *::before, *::after { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  color: #222;
  background: #fafafa;
}
@media (prefers-color-scheme: dark) {
  body { color: #eee; background: #111; }
}
```

The JavaScript entry point

```
const root = document.querySelector("#root");
async function load() {
  const res = await fetch("/api/items");
  const items = await res.json();
  root.innerHTML = items
    .map(i => `<li>${i.name}</li>`)
    .join("");
}
load();
```

Where to go next

Build this. Deploy it. Add a form, a fetch, a route, a bit of state. Read the source of one open-source app you admire. Then pick a framework: React if you want the largest ecosystem, Svelte if you want the simplest mental model, Vue if you want a balance of the two.

Closing Notes

If you have read this far, you have everything you need to start building real web pages and to read other people's code without getting lost. The web is enormous; this book is a foothold, not the summit.

What to learn next

- Build at least one small project end to end.
- Read the source of a popular open-source component library.
- Skim the MDN docs for elements and properties you have never used.
- Learn one frontend framework deeply (React, Vue, or Svelte).
- Pick up a backend; even a tiny Node or Python server changes how you think about the frontend.

A final word

The best web developers are not the ones who know everything. They are the ones who know the small set of core ideas well enough to read documentation quickly, recognize patterns, and ship something every week. You now know those core ideas. The rest is practice.

Glossary

A short reference of terms used throughout this book. Each entry is one sentence: just enough to recognize the word when you encounter it in the wild.

API

Application Programming Interface. A defined surface a piece of code exposes to other code.

ARIA

Accessible Rich Internet Applications. A spec for adding accessibility metadata to HTML.

Async

An operation that does not complete immediately; the caller continues while it runs in the background.

Attribute

Extra information attached to an HTML tag, written as `name="value"`.

Bubbling

The default propagation of DOM events from the target up through every ancestor.

Cache

A place where previously fetched resources are stored for fast reuse.

Callback

A function passed to another function so it can be invoked later.

Cascade

The CSS algorithm that decides which conflicting rule wins for a given property on a given element.

Closure

A function bundled with a reference to the variables it had access to when created.

CORS

Cross-Origin Resource Sharing. The server-side mechanism that lets the browser allow cross-origin requests.

CSP

Content Security Policy. A response header that restricts which resources a page may load.

DOM

Document Object Model. The browser's in-memory tree representation of HTML.

Event Loop

The runtime mechanism that decides which queued task or microtask runs next.

Fetch

The modern browser API for making HTTP requests from JavaScript.

Flexbox

A CSS layout system for one-dimensional arrangements of items along a row or column.

Grid

A CSS layout system for two-dimensional arrangements of items into rows and columns.

Hoisting

The way function declarations and var bindings appear to be moved to the top of their scope.

HTTP

HyperText Transfer Protocol. The request-response protocol the web is built on.

HTTPS

HTTP layered over TLS. Encrypts traffic between browser and server.

JSON

JavaScript Object Notation. The dominant text format for data interchange on the web.

Module

A file with its own scope that exports values for other modules to import.

Pixel Pipeline

The five-step process by which the browser turns code into pixels: parse, style, layout, paint, composite.

Polyfill

Code that adds a missing feature to older browsers so newer code runs unmodified.

Promise

An object representing a value that will exist eventually, with `.then` and `.catch` methods.

Pseudo-class

A CSS selector that targets elements in a particular state, such as `:hover` or `:focus`.

Reflow

Another name for the browser's layout step; usually expensive.

Selector

The part of a CSS rule that picks which elements the rule applies to.

Semantic HTML

Using HTML tags that describe the meaning of content, not its appearance.

Specificity

The score that ranks competing CSS selectors when the cascade resolves a conflict.

Throttle

Limit how often a function may be called over time; often used for scroll or resize handlers.

Viewport

The visible area of the browser window.

XSS

Cross-Site Scripting. The class of attacks that inject untrusted scripts into a page.

Box Model

The four-layer geometry of every element: content, padding, border, margin.

Breakpoint

A screen width at which a layout switches from one arrangement to another.

Cookie

A small key-value pair stored by the browser and sent on every request to its origin.

Defer

An attribute on `<script>` that downloads in parallel but runs after the document parses.

Em / Rem

CSS units relative to the element's font size and to the root font size, respectively.

Hydration

The process of attaching JavaScript behavior to server-rendered HTML.

Iterable

Any object that exposes a `Symbol.iterator` method so it can be used with `for-of` and `spread`.

Landmark

A semantic HTML region (header, nav, main, footer) that assistive tech exposes for quick navigation.

LCP

Largest Contentful Paint. The Core Web Vital that measures perceived load speed.

Microtask

A task queued by Promises that runs before the next macrotask.

Origin

The combination of scheme, host, and port that defines a security boundary in the browser.

Repaint

The browser's paint step, redrawing pixels without a fresh layout pass.

Service Worker

A background script that intercepts network requests to enable offline behavior.

Shadow DOM

An encapsulated DOM subtree, used by web components to scope styles and markup.

Source Map

A file mapping minified production code back to its original source for debugging.

TLS

Transport Layer Security. The cryptographic protocol underneath HTTPS.

Tree Shaking

A bundler optimization that removes exports never imported anywhere.